



GENERALIZED ESTIMATION SYSTEM

Version 2.03

Calibration Module – Methodology Guide

October 2019

G-Est

Table of Contents

1	Introduction	4
1.1	Notation	5
2	Calibration.....	6
2.1	The Basic Calibration Problem	7
2.1.1	Performing Calibration	7
2.1.2	Calibration within selected groups	8
2.1.3	Remarks	8
2.1.4	Example of distances used for calibration	10
2.1.5	General Remarks.....	11
2.1.6	Implementation in G-Est.....	13
2.2	Calibration for Two-Phase Sampling	15
2.2.1	Two-Phase Sampling	15
2.2.2	Three types of auxiliary variables	17
2.2.3	Calibration Weights in a Two-Phase Design	18
2.2.4	Implementation in G-Est.....	20
2.3	Calibration for Cluster Sampling and Weight Integration	22
2.3.1	Auxiliary information at the element level only	23
2.3.2	Auxiliary information at the cluster level only.....	25
2.3.3	Auxiliary information at both the cluster and element levels	26
2.4	Calibrating Bootstrap Replicate Weights	26
2.4.1	Handling non-positive entry bootstrap replicates values aib	27
2.4.2	Handling Special Cases.....	28
2.5	Special Features and Exceptions	29
2.5.1	Tolerance Parameter	29
2.5.2	Fixed weights – Not to Calibrate Sets	29
2.5.3	Small Sample Size (Contributing Terms)	30
3	Using OPTMODEL in SAS	32
3.1	Quadratic Programming.....	32
3.2	Interior Point Algorithm	32
3.2.1	Excerpts from SAS User guide.....	32
	REFERENCES.....	35

APPENDIX A (macro call).....	36
List of parameters with short description	37
APPENDIX B	40
B1 Using Single-Stage Calibration to Perform Two-Stage Calibration With Integrated Weighting.....	40
B2 Calibration (Household – Person – Kish individual).....	42

1 Introduction

This document presents a summary of the methodology behind calibration and provides a description of **Gest_Calibration**; the calibration module for version 2.03 of G-Est. The module **Gest_Calibration** of G-Est computes calibration weights for one and two-phase designs, covers calibration in the case of (single-stage) cluster sampling and performs weight integration. It also handles calibration of bootstrap replicates, which is calibrating more than one column of weights. It takes advantage of the use of auxiliary variables and can handle general calibration groups. These groups do not have to be mutually exclusive or exhaustive. Each group can have different auxiliary variables or no auxiliary variables at all. Also, the function allows the user to specify bounds on the calibration weights or the calibration factors (g-factors) to be computed. Parameter description together with different features are given in the user guide: *G-Est 2.03 - User Guide - Calibration.pdf*.

[Section 2](#) of this document gives an overview of the calibration procedure and how it is implemented in the module **Gest_Calibration**. To calibrate, one needs to have the values of one or more auxiliary variables for each unit in the sample and must know the corresponding auxiliary totals over the associated target population for each calibration group. These totals could be estimated as in a two-phase calibration procedure, for instance.

[Section 2.1](#) covers the case of a simple one stage one phase calibration. Two-phase calibration is treated in [section 2.2](#). Calibration and weight-integration in the case of (single-stage) cluster sampling is covered [section 2.3](#). The use of auxiliary information at both the cluster (enterprise/household) and the element (establishment/individual) levels is described. The module offers a number of special features that are listed in [section 2.4](#).

The module uses both the quadratic programming (QP) solver and the nonlinear programming (NLP) solver of the OPTMODEL procedure in SAS to solve the calibration minimization program. Both solvers use the so called *Interior Point Algorithm*. The default solver being QP and NLP is called if QP fails to establish convergence due, for example, to numerical problems. This is described in [section 3](#).

With some pre-processing, the module could also be used to perform calibration for the following designs/cases:

- Multi-stage designs, see [appendix B1](#).
- The “Household – person – Kish individual” design, see [appendix B2](#).

1.1 Notation

The following notation will be used in this document:

a_k	Design or starting weight for element k .
c_k	(Regression) Adjustment factor associated with element k .
G_k	A pseudo distance.
g_k	Calibration factor ($g_k = w_k/a_k$) for element k .
k	An element/unit of a sample s or a population U .
L	Lower bound constraint on calibration factors $g_k = w_k/a_k$.
l_k	Lower bound constraint on the calibration weight w_k .
M	Upper bound constraint on calibration factors $g_k = w_k/a_k$.
N	Size (number of elements) of the target population U .
n	Size of the sample $s \subseteq U$.
π_k	Probability of inclusion of element k in the sample.
π_{1k}	Phase1 inclusion probability of element k in the first phase sample s_1 : $\pi_{1k} = Pr(k \in s_1)$.
π_{2k}	$= \pi_{k s_1} = Pr(k \in s k \in s_1)$ phase2 (conditional) inclusion probability for element $k \in s_1$.
π_k^*	$= \pi_{1k} \pi_{k s_1} = \pi_{1k} \pi_{2k}$.
s	A sample of elements selected using a sampling design. Also denotes the phase-two sample.
T_y	Population total for variable y .
$\hat{T}_y$	An estimator of a total T_y .
$\hat{T}_y^{CAL}$	Calibration estimator of T_y .
T_x	$= \sum_U \mathbf{x}_k$, population total for the auxiliary vector $\mathbf{x}$.
U	The population of elements from which a sample s is drawn.
u_k	Upper bound constraint on the calibration weight w_k .
w_k	Calibration weight for element k .
$\mathbf{x}_k$	Auxiliary vector associated with element k .
y	A variable of interest.
y_k	The value of the variable of interest y for element k .

2 Calibration

Calibration is a weight adjustment technique that consists of modifying initial weights (design weights or other) to force the estimates from the survey to be consistent with totals known from another reliable source (censuses, administrative data, other surveys, etc.). This technique takes advantage of the use of auxiliary information.

Set-Up

Let $U = \{1, 2, \dots, k, \dots, N\}$ be a finite population of size N and suppose we would like to estimate a certain unknown parameter, say a total T_y , for a variable of interest y , revenue for instance. Let $s \subseteq U$ be a probability sample of size $n \leq N$, $\pi_k = P(k \in s)$, the selection probability for element $k \in U$ and y_k the value of y for element k . The π_k are supposed to be positive for all $k \in U$; $\pi_k > 0$. The population total $T_y = \sum_{k \in U} y_k \equiv \sum_U y_k$ is then estimated, for example, by the so-called expansion estimator $\hat{T}_y = \sum_s y_k / \pi_k = \sum_s a_k y_k$ also known as the Horvitz-Thompson estimator. This estimator is design unbiased, which is a desirable property. The quantity $a_k = 1 / \pi_k$ is called the sampling or design weight for element k (in the sample). These weights do not depend on the survey variable for which we want to estimate a total, mean or other parameter of interest. In fact, they could be used to estimate the population total of any survey variable.

Deville and Särndal (1992) coined the term “calibration estimator” to describe an estimator of the form $\hat{T}_y^{CAL} = \sum_s w_k y_k$, where w_k is such that $\sum_s w_k \mathbf{x}_k = \sum_U \mathbf{x}_k \equiv T_{\mathbf{x}}$ (benchmark constraint) for some vector $\mathbf{x}_k$ about which the total $T_{\mathbf{x}}$ is known. The weights are obtained by minimizing a distance between the sought-after vector $(w_1, w_2, \dots, w_n)'$ and $(a_1, a_2, \dots, a_n)'$ under the above benchmark constraint.

Reasons for Calibration

Usually, we have auxiliary variables (information) available from censuses, administrative data, other surveys, etc. That is, we know the value of a vector $\mathbf{x}_k$ for $k \in s$, or even for $k \in U$ and the corresponding population total: $T_{\mathbf{x}} = \sum_U \mathbf{x}_k$. However, in general, $\hat{T}_{\mathbf{x}} = \sum_s a_k \mathbf{x}_k \neq T_{\mathbf{x}}$, which creates an inconsistency between survey and population totals (known or widely accepted as being accurate). A solution to this inconsistency problem consists of modifying the design weights so that survey estimates of the quantities of interest such as totals (counts or means) coincide with true, known totals (counts or means) from external sources (censuses, administrative data, etc.). We thus have two constraints here:

- New weights have to be as close as possible (through the use of a distance measure) to the old ones to obtain an estimator that is “approximately” unbiased (desired statistical property).
- The new weights have to give very good estimates of known quantities (or widely accepted as being accurate.)

This procedure is called calibration.

Note

Design weights a_k are sometimes modified before calibration procedure takes place

$$a_k \xrightarrow{\text{adjustment for nonresponse or other}} b_k \xrightarrow{\text{calibration}} w_k$$

Some of the reasons why we do calibration are:

- Coherence between estimates from the survey and known totals in the population or widely accepted as being accurate.
- Reduce non-sampling errors such as non-response errors and coverage errors.
- Improve the precision (variance) of the estimators.

2.1 The Basic Calibration Problem

2.1.1 Performing Calibration

As we saw before, calibration is about finding a new set of weights called calibration weights as close as possible to the old ones (design weights or other) satisfying some benchmark constraints. Therefore, we need to solve a minimization program like the following one:

Minimize G with respect to w_k	$G = \sum_{k \in S} G_k(w_k, a_k) = \sum_{k \in S} \frac{c_k (w_k - a_k)^2}{a_k}$
Subject to	Benchmark constraints (calibration equations): $\sum_{k \in S} w_k \mathbf{x}_k = \sum_U \mathbf{x}_k$. Range restriction constraints: $l_k \leq w_k \leq u_k \quad \text{or} \quad L \leq \frac{w_k}{a_k} \leq M \quad (L, M \text{ known constants.})$

where

- G : a (pseudo) distance (here the so called Chi-square distance),
- $a_k, k \in s$: Initial/starting/base weights (design or other), $a_k > 0$.
- $w_k, k \in s$: the required calibration weights,
- $c_k, k \in s$: adjustment factor (importance of element k), must be positive,
- $\sum_{k \in S} w_k \mathbf{x}_k = \sum_U \mathbf{x}_k$: **calibration equations**, one for each auxiliary variable, which could be rendered as a set of $p \geq 1$ univariate calibration equations:

$$\sum_{k \in S} w_k x_{jk} = \sum_{k \in U} x_{jk}, \quad j = 1, 2, \dots, p.$$

- $\mathbf{x}_k = (x_{1k}, \dots, x_{pk})^T$, $p \geq 1$; vector of auxiliary variables,
- $l_k, u_k, k \in s$: lower and upper bound constraints on the calibration weights,
- L, M : lower and upper bound constraints on the calibration factors $g_k = \frac{w_k}{a_k}$, $a_k > 0$.

Remarks

Calibration equations could be written in matrix format as follows $\mathbf{A} \mathbf{w} = \mathbf{X} = \sum_U \mathbf{x}_k$, where

$$\mathbf{A} = \begin{pmatrix} x_{11} & x_{12} & \dots & x_{1n} \\ x_{21} & x_{22} & \dots & x_{2n} \\ \vdots & \vdots & \dots & \vdots \\ x_{p1} & x_{p2} & \dots & x_{pn} \end{pmatrix} = (\mathbf{x}_1 \dots \mathbf{x}_k \dots \mathbf{x}_n), \quad \mathbf{w} = \begin{pmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{pmatrix}, \quad \mathbf{X} = \sum_U \mathbf{x}_k = \begin{pmatrix} \sum_{k \in U} x_{1k} \\ \sum_{k \in U} x_{2k} \\ \vdots \\ \sum_{k \in U} x_{pk} \end{pmatrix}$$

- l_k, u_k could be equal to $-\infty, +\infty$ respectively (in general, we look for positive weights).
- Calibration equations are usually **within groups**. One might have known totals within provinces, for instance, so we would have calibration equations for each province. We will write:

$$\sum_{k \in S} w_k \mathbf{x}_k g_k = \sum_U \mathbf{x}_k g_k, \text{ where } g_k = I(k \in g) = 1 \text{ if } k \in \text{group } g, \text{ and } 0, \text{ otherwise.}$$
- In general, we impose the following restriction on the factors bounds: $0 < L \leq 1 \leq M$.

2.1.2 Calibration within selected groups

In general, calibration is done within groups, called *calibration groups*. So the user might want to calibrate by province or province and Naics¹ for example. In that case he/she has to provide the known totals for the corresponding group. The minimization problem would be written as follows:

Minimize G with respect to w_k	$\sum_{k \in S} G_k(w_k, a_k) = \sum_{k \in S} \frac{c_k (w_k - a_k)^2}{a_k}$
Subject to	$\sum_{k \in S} w_k \mathbf{x}_k g_k = T_{\mathbf{x}, g} = \sum_U \mathbf{x}_k g_k, \quad g = 1, 2, \dots, G.$
	$l_k \leq w_k \leq u_k \quad \text{or} \quad L \leq \frac{w_k}{a_k} \leq M \quad (L, M \text{ known constants.})$

where $g_k = I(k \in g)$ is the indicator for the group, that is, $g_k = 1$ if $k \in g$, and zero otherwise. The total $T_{\mathbf{x}, g}$, is the total of the auxiliary variable $\mathbf{x}$ (say revenue) for group g .

2.1.3 Remarks

Calibration groups do not have to be mutually exclusive or exhaustive. Each group can have different auxiliary variables or no auxiliary variables attached to it. For instance one might have known totals for some provinces only or some Naics only. Let $\mathbf{x} = (x_1, x_2)^T$ (revenue and expense, for example) and we want to calibrate by groups g_1 (province) for x_1 and g_2 (Naics) for x_2 . In this case the benchmark constraints are given by

¹ North American Industry Classification System
 G-Est 2.03 - Methodology Guide – Calibration

$$\sum_{k \in s} w_k x_{1k} g_{1k} = \sum_U x_{1k} g_{1k}, \quad g_1 = 1, 2, \dots, G_1. \quad g_{1k} = I(k \in g_1).$$

$$\sum_{k \in s} w_k x_{2k} g_{2k} = \sum_U x_{2k} g_{2k}, \quad g_2 = 1, 2, \dots, G_2. \quad g_{2k} = I(k \in g_2).$$

Here G_1 is the number of provinces, although the user could provide totals for some provinces (groups) only.

We could have different calibration groups for different auxiliary variables or for a given auxiliary variable. For instance, one might want to calibrate to counts by province (all or part) and by Naics (all or part.)

The function **Gest_Calibration** allows the user to specify calibration groups of different forms for given auxiliary variables for which a total is available. This is done using the two columns `CalibrationGroup` and `CalibrationGroupExpression` in the input SAS file *CalibrationGroupFile* (see user guide for more details). Some examples are given in the next table:

	Calibration Group		Auxiliary variable
	Calibrate by (<code>CalibrationGroup</code>)	where statement (<code>CalibrationGroupExpression</code>)	
(1)	province	Province $\notin (P1, P2)$	revenue
(2)	Province*Naics		count
(3)	strata	Size $\neq$ 'LARGE'	revenue
(4)	strata	DesignWeight $\neq$ 1	expense

In (1), we calibrate by province. We have total revenue for all provinces except for provinces P1 and P2 (or we do not want to modify the weights for units in these two provinces.) So we will have a calibration equation for each province except for P1 and P2.

In (2), we calibrate by PROVINCE*NAICS to known (or estimated) total counts.

In (3), we calibrate by variable strata using auxiliary variable revenue and we want to modify the weights of all sample units except for large companies, example. Population totals by strata (for non-large companies) are supposed to be available.

In (4), we have population totals ('without the take-alls') of variable expense and we would like to calibrate using calibration variable strata. However, we do not wish to modify the weights for take-alls. One might, for instance, calibrate by province but does not want to modify the weights for take-alls. The totals could be estimated ones as well.

There exists in **Gest_Calibration** a parameter called `CalibrationExclusionExpression`, that allows the user to exclude a subset of sample units from the calibration procedure, see user guide for more details.

2.1.4 Example of distances used for calibration

Examples of distances used in the literature are (see for instance Deville and Särndal, 1992):

Table 1

Distance: $c_k^{-1} G_k(w_k, a_k)$		Properties
1	Chi-square distance $\frac{(w_k - a_k)^2}{2a_k}, \quad w_k \in \mathbb{R}.$	Popular! Easy, fast, “converges”. Gives the GREG estimator. Possible negative weights
2	Truncated Chi-square $\begin{cases} \frac{(w_k - a_k)^2}{2a_k}, & \text{if } L \leq \frac{w_k}{a_k} \leq U, \\ \infty, & \text{otherwise.} \end{cases}$	Avoids negative, zero or large weights Usual choices: .5 ≤ L ≤ .8; 1.2 ≤ U ≤ 2.5
3	Raking Ratio (Kullback-Leibler information) $w_k \log\left(\frac{w_k}{a_k}\right) - w_k + a_k$	Produces always positive weights Extreme large values possible Raking
4	Logit $\begin{cases} \left[A_k \log\left(\frac{A_k}{1-L}\right) + B_k \log\left(\frac{B_k}{U-1}\right) \right] \frac{a_k}{A}, & \text{if } L \leq \frac{w_k}{a_k} \leq U, \\ \infty, & \text{otherwise.} \end{cases}$ $A_k = \frac{w_k}{a_k} - L, \quad B_k = U - \frac{w_k}{a_k}, \quad A = \frac{U-L}{(1-L)(U-1)}$	Avoids negative, zero or large weights Bounded weights
5	Roy and Vanheuverzwyn (hyperbolic Sin function) $a_k \int_1^{w_k/a_k} \frac{1}{2\alpha} \sinh\left[\alpha\left(t - \frac{1}{t}\right)\right] dt$	Positive weights No extreme values (reduced problem!) Bounds on weights using α

2.1.5 General Remarks

2.1.5.1 Handling negative initial weights

For some units, initial weights could be zero or even negative due to previous adjustments including calibration. If a_k is smaller than some threshold $\varepsilon > 0$ then in the distance function a_k is replaced by $\max(a_k, \varepsilon)$ ($\varepsilon = 10^{-6}$, for instance) and calibration is performed. It is optional to give a special treatment when $a_k = 0$ so that it does not participate in the distance function. **So negative values are replaced by a (“very small”) positive number in the distance function but not in the calibration equations.** This is a methodological choice. Another choice would be to force negative initial weights to take the value 1 (or other), for instance, and then calibrate.

New Chi-Square distance

$$G = \sum_{k \in S} \frac{c_k [w_k - \max(a_k, \varepsilon)]^2}{\max(a_k, \varepsilon)}$$

or

$$G = \sum_{k \in S - \{a_k=0\}} \frac{c_k [w_k - \max(a_k, \varepsilon)]^2}{\max(a_k, \varepsilon)}.$$

New Raking Ratio distance

$$G = \sum_{i \in S} c_k \left[w_k \log \left(\frac{w_k}{\max(a_k, \varepsilon)} \right) - w_k + \max(a_k, \varepsilon) \right]$$

or

$$G = \sum_{i \in S - \{a_k=0\}} c_k \left[w_k \log \left(\frac{w_k}{\max(a_k, \varepsilon)} \right) - w_k + \max(a_k, \varepsilon) \right].$$

Remark

- When $a_i^{(b)} > 0$, we obtain the usual distances programmed in Gest_Calibration version 1.
- We do not need to modify the values of the $a_i^{(b)} \leq 0$ in the SampleFile. So initial/entry weights (DesignWeightVariable) could be negative or zero in theory. However they are handled by the objective function (and in the range restriction constraints for the factors).

Note

The user has to be cautious here in interpreting the factors when initial/entry weights are zero or negative.

2.1.5.2 The Distance

- It is possible to put bounds on any given distance.
- Two distances are implemented in version 2 of G-Est; the chi-square and the raking-ratio distances. Different range restriction constraints are allowed, see the user guide for more details. These constraints could be on the weights themselves or on the g-factors.
- When the sample size is large enough, all distances are then equivalent to the Chi-square distance when it comes to the calibration weights, see for example, Deville and Särndal (1992).

2.1.5.3 Special case (regression estimator weights)

- In the case of the Chi-square distance, we obtain, assuming some regularity conditions, the following well-known solution (here we assume no range constraints)

$$\begin{aligned} w_k &= a_k \left\{ 1 + c_k^{-1} (T_x - \hat{T}_x)' \left[\sum_{j \in S} c_j^{-1} a_j \mathbf{x}_j \mathbf{x}_j' \right]^{-1} \mathbf{x}_k \right\} \\ &= a_k + (T_x - \hat{T}_x)' \left[\sum_{j \in S} c_j^{-1} a_j \mathbf{x}_j \mathbf{x}_j' \right]^{-1} c_k^{-1} a_k \mathbf{x}_k = a_k \times g_k. \end{aligned}$$

- The quantities $g_k = w_k/a_k$ are called **calibration factors**.
- Calibration weights could be **negative** (or zero).
- With the above solution, we do obtain the equality $\sum_{k \in S} w_k \mathbf{x}_k = \sum_U \mathbf{x}_k$.
- Calibration weights **depend on the sample**; $w_k = w_k(s)$.

2.1.5.4 Calibration Estimator and its Properties

The calibration estimator is obtained by using calibration weights. It takes the following form $\hat{T}_y^{CAL} = \sum_{k \in S} w_k(s) y_k$, where $w_k(s)$, is the calibration weight. In the case of the Chi-square distance and under no range restrictions, the calibration estimator takes the form

$$\hat{T}_y^{CAL} = \sum_{k \in S} w_k(s) y_k = \hat{T}_y + (T_x - \hat{T}_x)' \left[\sum_{k \in S} c_k^{-1} a_k \mathbf{x}_k \mathbf{x}_k' \right]^{-1} \sum_{k \in S} c_k^{-1} a_k \mathbf{x}_k y_k$$

When the sample size is large enough we have the following properties (see Deville and Särndal (1992)):

- calibration estimator is approximately unbiased,
- all calibration estimators have the same asymptotic variance whatever the distance used,
- all distances are then equivalent to the Chi-square distance,
- whatever distance is used, the calibration estimator is equivalent to the one using the Chi-square distance.

2.1.6 Implementation in G-Est

The module **Gest_Calibration** performs one and two-phase calibration. The macro call and short parameter description is given in [appendix A](#). To perform one-phase calibration, one has to assign the value 1 to the parameter *CalibrationPhases* and provide the necessary calibration information in Block1. Please refer also to the Calibration User Guide for detailed information about the macro call and the role of each parameter.

The parameter *CalibrationGroupFile* points to a SAS data file containing the necessary information to help build the calibration equations. In this file, the user specifies the calibration groups, the auxiliary variables and the totals file containing the corresponding auxiliary totals. Also, a tolerance parameter could be provided. This parameter serves to relax the calibration constraints if desired.

So **Gest_Calibration** helps solve the following general minimization program:

Minimize with respect to w_k Subject to	$\sum_{k \in S} G_k(w_k, a_k) = \sum_{k \in S} \frac{c_k(w_k - a_k)^2}{a_k}$ $- \sum_{k \in S} w_k x_k g_k = T_g, \quad g = 1, 2, \dots, G.$ $- l_k \leq w_k \leq u_k \quad \text{or} \quad l \leq w_k \leq u \quad \text{or} \quad L \leq \frac{w_k}{a_k} \leq M.$
---	---

Note

It is possible to relax some or all the benchmark constraints (equality constraints) with the use of the parameter tolerance specified in the SAS data file *CalibrationGroupFile*. In that case, the calibration equations to be solved are given by:

$$\left| \frac{\sum_{k \in S} w_k x_k g_k - T_g}{T_g} \right| \leq \varepsilon \quad \Leftrightarrow \quad T_g - \varepsilon |T_g| \leq \sum_{k \in S} w_k x_k g_k \leq T_g + \varepsilon |T_g|, \quad T_g \neq 0.$$

$$\Leftrightarrow \quad (1 - \varepsilon)T_g \leq \sum_{k \in S} w_k x_k g_k \leq (1 + \varepsilon)T_g, \quad \text{if } T_g > 0.$$

Please refer to the section [special features](#) for other special features and exceptions. Here is a summary of different features available in **Gest_Calibration**.

2.1.6.1 Summary Features of Gest_Calibration

- One and two-phase calibration.
- Cluster calibration and weight integration.
- Allows the use of two possible distances.
- General calibration groups. These groups do not have to be mutually exclusive or exhaustive. Each group can have different auxiliary variables or no auxiliary variables attached to it.
- The module allows the user to specify bounds on the calibration weights and/or the calibration factors (g-factors) to be computed. Individual and uniform bounds are allowed.

- It is possible to specify a subset of weights for which one does not want to calibrate; take-all units, for instance. This is achieved by using the global parameter *CalibrationExclusionExpression*.
- It is possible to relax some or all benchmark constraints (equality constraints) using a parameter called *tolerance*. The latter is specified in the file *CalibrationGroupFile*.
- The module can exclude equations from calibration procedure if these equations involve missing values for the auxiliary variables, calibration groups, etc.
- It is possible to exclude calibration equations that have a “small” number of terms. This is done using parameter *SmallSampleSizeCalibration*. So calibration groups (categories) containing a small number of units are excluded from calibration procedure.
- Bootstrap weights calibration. For a given set of $B \geq 1$ (bootstrap) replicate weights, calibration is performed using the same sample file and calibration information. The calibrated input Bootstrap replicates are then saved into an output Bootstrap replicate file containing the CalibrationID (cluster id for instance) and the B calibrated Bootstrap replicates. A status file would be created as well. The latter consists of the entry status file which would be modified in the case of failure to calibrate.

2.1.6.2 What is new in version 2

- Allowing for negative or zero initial/entry weights (*i.e.* a_k). See user guide for more details.
- Possibility of specifying the range restriction constraints together, that is specifying: LowerWeightBound, UpperWeightBound, LowerFactorBound, UpperFactorBound and WeightBoundsFile. See [Appendix A](#) and the user guide for more details.
- Two distances; adding the raking ratio distance in version 2.
- Calibrating Bootstrap replicates.
- Specifying more than one variable in column AuxiliaryVariables in CalibrationGroupFile.
- Adding column *AuxiliaryVariablesLevel* in CalibrationGroupFile.
- Adding case #9 in ExclusionFile (contributing units), see parameter *SmallSampleSizeCalibration* and section *ExclusionFile and ExclusionFileBypass* in the user guide.

2.2 Calibration for Two-Phase Sampling

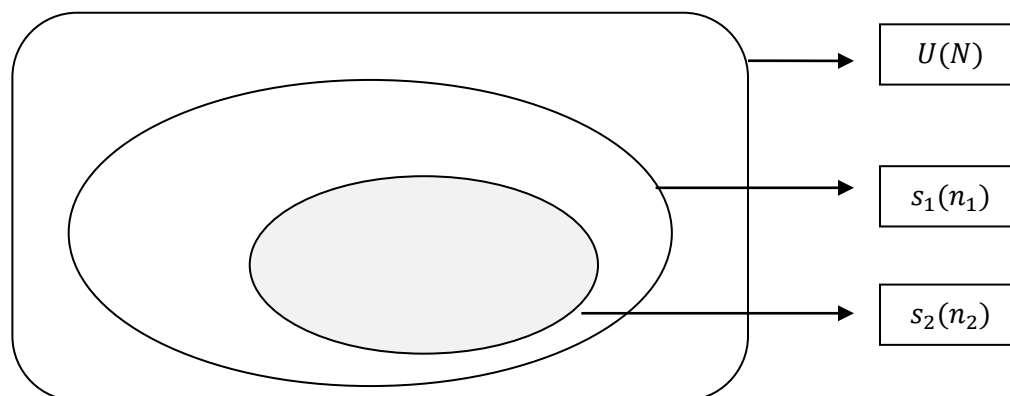
2.2.1 Two-Phase Sampling

The sample frame often contains little or no auxiliary information. In that case, two-phase sampling is a solution that permits the use of (collected) auxiliary information that will help improve quality of estimation in general. Two-phase sampling is a two-step procedure that consists of the following:

- At phase1: a first sample s_1 is selected from the population U according to a sampling design p_1 (or $p_1(s_1)$).
- At phase2: a subsample s_2 of s_1 is then selected according to a sampling design p_2 (or $p_2(s_2)$).

The first phase sample serves to gather inexpensive (auxiliary) information that will enhance the quality of estimations produced using the second phase sample s_2 .

Figure 1



Note. In what follows, s_2 is denoted by s , and n_2 by n .

Two-phase inclusion probabilities

- U : a finite population of size N .
- $T_y = \sum_U y_k$: the parameter of interest to be estimated.

Phase1

- A large sample $s_1 \subset U$ is selected according to a given design (might involve stratification)
- For elements in s_1 , auxiliary information is recorded that will permit stratification for phase2 sample selection.
- Let $\pi_{1k} = Pr(k \in s_1) > 0$, be the phase1 inclusion probability for element $k \in U$.

Phase2

- The recorded auxiliary information in phase1 is used to stratify the phase1 sample s_1 into G strata (called the phase2 strata) to give $s_1 = \bigcup_{g=1}^G s_{1g}$.
- From each stratum g , a sample $s_g \subseteq s_{1g}$ is selected to give the phase2 sample $s = \bigcup_{g=1}^G s_g$.
- Let $\pi_{2k} = \pi_{k|s_1} = Pr(k \in s | k \in s_1) > 0$, be the phase2 (conditional) inclusion probability for element $k \in s_1$.

Two-phase inclusion probability:

- Let $\pi_k = Pr(k \in s)$ be the two-phase design inclusion probability for element $k \in U$.
- π_k is, in general, difficult to calculate, so instead we use the following quantity $\pi_k^* = \pi_{1k} \pi_{k|s_1} = \pi_{1k} \pi_{2k}$. Note that, in general, $\pi_k^* \neq \pi_k$, unless $\pi_{k|s_1}$ is independent of s_1 , for example.

Two-phase design weights

To compute the weights, we use the calculated inclusion probabilities. The design weight that will be used for the two-phase design is the product of the phase-one weight and the second-phase weight.

- Phase1 design weights: $a_{1k} = 1 / \pi_{1k}, k \in s_1 \subseteq U$.
- Phase2 design weights: $a_{2k} = 1 / \pi_{2k}, k \in s \subseteq s_1 \subseteq U$.
- Two-Phase design weights: $a_k^* = \frac{1}{\pi_k^*} = \frac{1}{\pi_{1k} \pi_{k|s_1}} = \frac{1}{\pi_{1k} \pi_{2k}} = a_{1k} a_{2k}, k \in s$.

Example: SRSWOR / SRSWOR (Simple Random Sampling WithOut Replacement)

Phase1 design weights

$$a_{1k} = \frac{1}{\pi_{1k}} = \frac{N}{n_1}, \quad k \in s_1 \subset U, \quad size(U) = N, \quad size(s_1) = n_1.$$

Phase2 design weights

$$a_{2k} = \frac{1}{\pi_{2k}} = \frac{n_1}{n_2}, \quad k \in s \subset s_1 \subset U, \quad size(s) = n_2.$$

Design weights to be used for the two-phase design

$$a_k^* = a_{1k} a_{2k} = \frac{1}{\pi_{1k}} \frac{1}{\pi_{2k}} = \frac{N}{n_2}, \quad k \in s \subset s_1 \subset U.$$

The double expansion estimator

For two-phase sampling, an unbiased estimator of the total $T_y = \sum_U y_k$ is the so called *double expansion estimator*; it is given by the following expression: $\hat{T}_y = \sum_{k \in s} a_{1k} a_{2k} y_k$.

2.2.2 Three types of auxiliary variables

In order to obtain calibration weights for a two-phase design, auxiliary variables could be used at the following two levels (see, for example, Estevao and Särndal (2009)):

Auxiliary variables at the first sample level (s_1 – level)

Here we consider **three** possible types of auxiliary variables:

- $\mathbf{x}_{1k}$: vector of auxiliary variables known for every $k \in s_1$ (thus for every $k \in s$) with a known vector of totals $\mathbf{T}_{\mathbf{x}_1} = \sum_U \mathbf{x}_{1k}$. This vector might as well be known for all $k \in U$.
- $\mathbf{x}_{k(a)}$: vector of auxiliary variables known for every $k \in s_1$ and thus for every $k \in s$ with an unknown vector of totals $\mathbf{T}_{\mathbf{x}(a)} = \sum_U \mathbf{x}_{k(a)}$. This total will be estimated by $\hat{\mathbf{T}}_{\mathbf{x}(a)} = \sum_{s_1} a_{1k} \mathbf{x}_{k(a)}$, using the first phase design weights.
- $\mathbf{x}_{k(w)}$: vector of auxiliary variables known for every $k \in s_1$ and thus for every $k \in s$ with an unknown vector of totals $\mathbf{T}_{\mathbf{x}(w)} = \sum_U \mathbf{x}_{k(w)}$. This total will be estimated by $\hat{\mathbf{T}}_{\mathbf{x}(w)} = \sum_{s_1} w_{1k} \mathbf{x}_{k(w)}$, using first phase calibration weights w_{1k} .

Auxiliary variables at the second sample level only (s – level only)

$\mathbf{x}_2$: a vector of auxiliary variables observed for elements of s only with a known vector of totals: $\mathbf{T}_{\mathbf{x}_2} = \sum_U \mathbf{x}_{2k}$.

Note

In practice, calibration is done within groups and totals are known (or estimated as in a two-phase design) within these groups. So, for instance, $\mathbf{T}_{\mathbf{x}_1} = \sum_U \mathbf{x}_{1k}$, would be replaced by $\mathbf{T}_{\mathbf{x}_{1,g}} = \sum_U \mathbf{x}_{1k} g_k$, where $g_k = I(k \in g)$.

We will denote by $\mathbf{x}_{k(t)}$ the vector of auxiliary variables composed of all or part of the elements of $\mathbf{x}_{1k}$ and $\mathbf{x}_{2k}$ for $k \in s$. When calibrating at the second phase, the user might want to use all or part of the auxiliary variables for which he/she knows the totals in the population. She/he might, for example, decide to use only the vector $\mathbf{x}_2$ (or part of it), or use the full vectors $\mathbf{x}_1$ and $\mathbf{x}_2$ or part of $\mathbf{x}_1$ and part of $\mathbf{x}_2$. Here is a table summarising different sets of auxiliary variables:

Sets of auxiliary variables for calibration in two-phase sampling				
Sets of Auxiliary variables	Auxiliary variable	Auxiliary variable total over U	Unit variable values for $k \in s_1$	Unit variable values for $k \in s$
E_1	$\mathbf{x}_1$	known	known	known
E_a	$\mathbf{x}_a$	unknown	known	known
E_w	$\mathbf{x}_w$	unknown	known	known
E_2	$\mathbf{x}_2$	known	unknown (in part)	known
$E_t \subseteq E_1 \cup E_2$	$\mathbf{x}_t$	known	known $\mathbf{x}_1$ unknown $\mathbf{x}_2$	known

If $E_t = E_1 \cup E_2$, for example, then $\mathbf{x}_{k(t)} = (\mathbf{x}_{1k}, \mathbf{x}_{2k})$, the full vector is used.

2.2.3 Calibration Weights in a Two-Phase Design

Calibration weights for the phase2 sample are obtained as a result of a two-step procedure:

Step 1: we calculate intermediate (calibration) weights w_{1k} satisfying the following constraint:

$$\sum_{k \in s_1} w_{1k} \mathbf{x}_{1k} = \mathbf{T}_{\mathbf{x}_1} = \sum_U \mathbf{x}_{1k}.$$

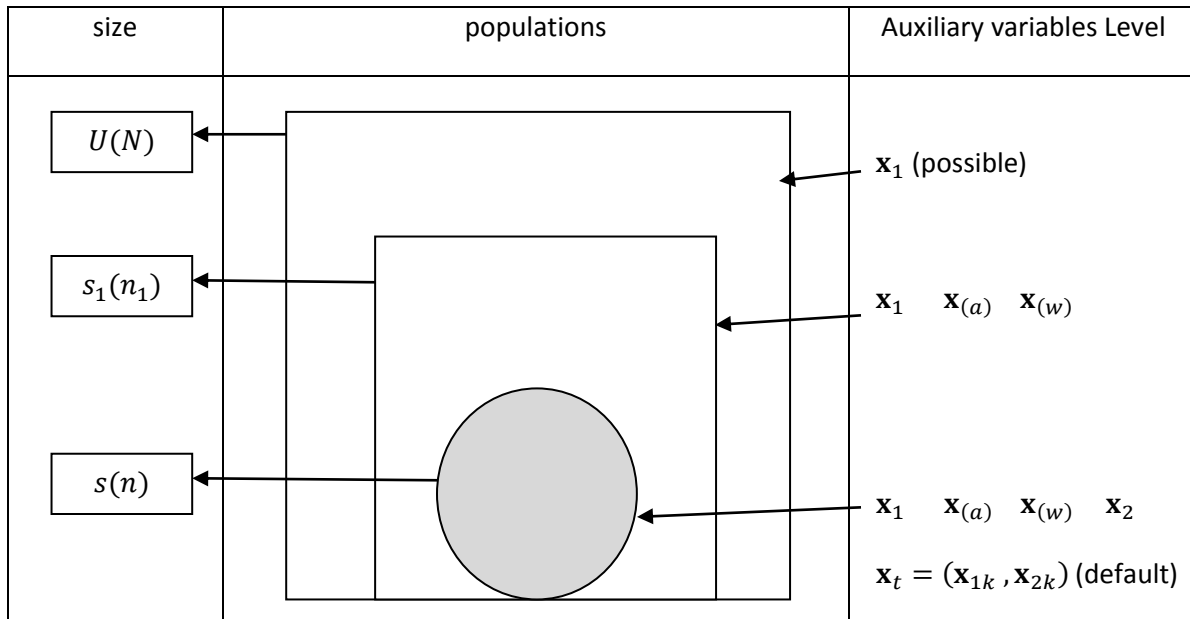
Step 2: use the intermediate weights w_{1k} to compute the final calibration weights w_k to satisfy

$$\sum_{k \in S} w_k \mathbf{x}_k = \sum_{k \in S} w_k \begin{pmatrix} \mathbf{x}_{k(t)} \\ \mathbf{x}_{k(w)} \\ \mathbf{x}_{k(a)} \end{pmatrix} = \begin{pmatrix} \mathbf{T}_{\mathbf{x}(t)} \\ \hat{\mathbf{T}}_{\mathbf{x}(w)} \\ \hat{\mathbf{T}}_{\mathbf{x}(a)} \end{pmatrix} = \begin{pmatrix} \sum_U \mathbf{x}_{k(t)} \\ \sum_{s_1} w_{1k} \mathbf{x}_{k(w)} \\ \sum_{s_1} a_{1k} \mathbf{x}_{k(a)} \end{pmatrix} \equiv \mathbf{T}.$$

Note

The calibration module **Gest_Calibration** allows the user to use all or part of the auxiliary variables at hand. One might even decide not to calibrate at step/phase 2 and work with $w_k = w_{1k}a_{2k}$ or even with $w_k = a_{1k}a_{2k}$ (no calibration at all!) as the final weights. It is possible to calibrate at both phases, at one of them only or skip calibration. In that case calibration weights are set to be the entry/design weights; $w_k = a_{1k}a_{2k}$. This could be useful if one needs to calculate intermediate totals using some given calibration groups for use at a later-stage! Furthermore, the output calibration file becomes an input file for a variance module.

Figure 2



2.2.3.1 Phase-One Calibration

In this phase, we try to find calibration weights w_{1k} that are solutions of the following minimization problem

Minimize	$\sum_{k \in s_1} G_k(w_{1k}, a_{1k}) = \sum_{k \in s_1} \frac{c_{1k}(w_{1k} - a_{1k})^2}{a_{1k}}, \text{ with respect to } w_{1k}$
Subject to	$- \sum_{k \in s_1} w_{1k} \mathbf{x}_{1k} g_k = \sum_U \mathbf{x}_{1k} g_k, \quad g = 1, 2, \dots, G.$ $- l_{1k} \leq w_{1k} \leq u_{1k} \quad \text{or} \quad L_1 \leq \frac{w_{1k}}{a_{1k}} \leq M_1 \quad (L_1, M_1 \text{ known constants.})$

where $g_k = I(k \in g)$ is the indicator for the group, that is, $g_k = 1$ if $k \in g$, and zero otherwise.

Note

If there are no auxiliary variables available at phase1 or if no auxiliary information is used, we then define the calibration weights to be the design weights, that is, $w_{1k} = a_{1k}$, for all $k \in s_1$.

2.2.3.2 Phase-Two Calibration

For phase2 calibration we are looking for a set of calibration weights w_k that are solutions of the following minimization problem

Minimize	$\sum_{k \in S} G_k(w_k, a_k^*) = \sum_{k \in S} \frac{c_k(w_k - a_k^*)^2}{a_k^*}, \text{ with respect to } w_k$
Subject to	$- \sum_{k \in S} w_k \mathbf{x}_k \equiv \mathbf{T}$ $- l_k \leq w_k \leq u_k \quad \text{or} \quad L \leq \frac{w_k}{a_k^*} \leq M \quad (L, M \text{ known constants.})$

Remarks

- The starting weight for phase2 calibration is given by either of the two following expressions

$$a_k^* = a_{1k} a_{2k} = \frac{1}{\pi_k^*} = (\pi_{1k} \pi_{k|s_1})^{-1} \text{ (This is the choice for version 2 of G-Est), or by}$$

$$a_k^* = w_{1k} a_{2k} = w_{1k} (\pi_{k|s_1})^{-1}, \text{ (using phase1 calibration weights). In this case the entry weight for phase2 could be negative or zero if } w_{1k} \text{ is.}$$

- One might as well choose his/her own starting weights for phase2. One choice would be to multiply one of the starting points in (1) by an adjustment factor before entering the procedure.

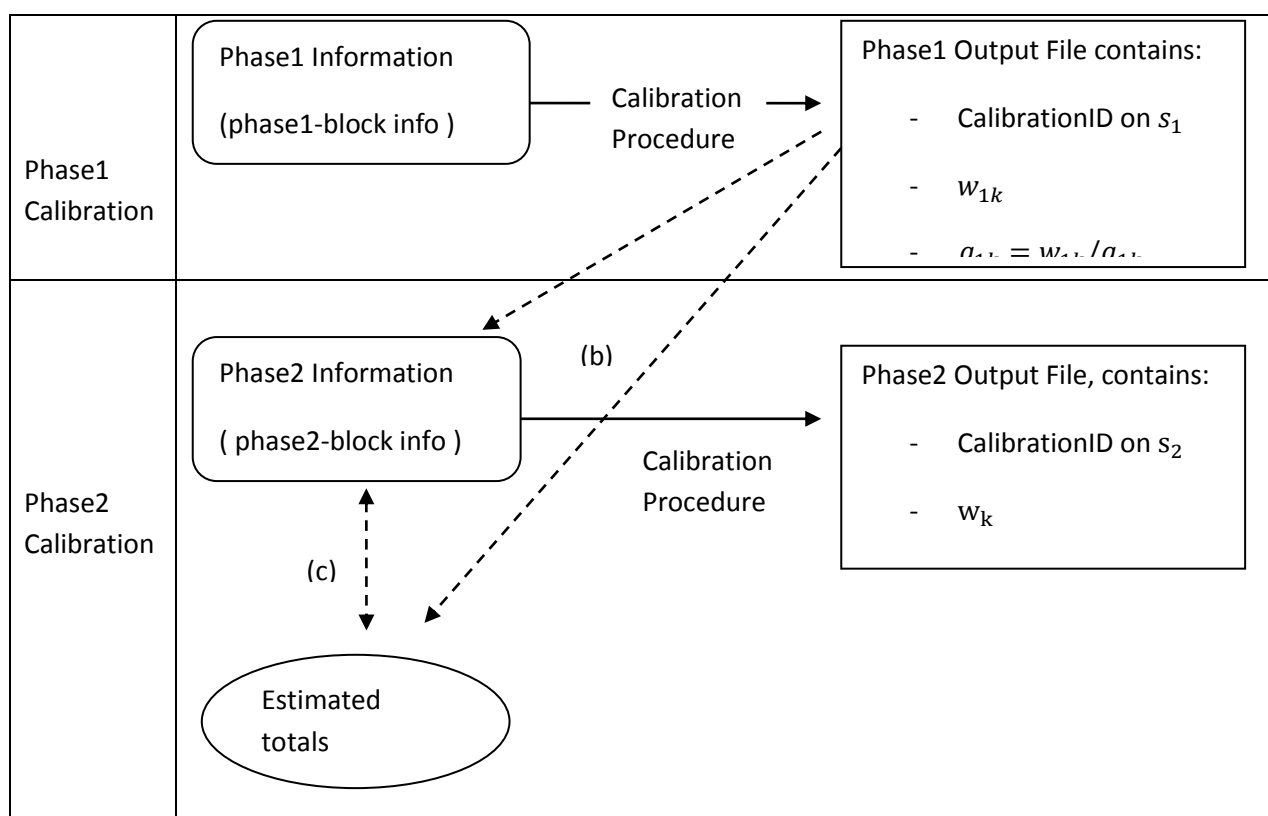
See Estevao and Särndal (2009), section 4 for a comparison of the two starting options above.

2.2.4 Implementation in G-Est

To perform two-phase calibration with **Gest_Calibration**, one has to provide the necessary information in blocks 1 and 2 in the macro call, see [appendix A](#). There are three stages here (see diagram 1 below):

- **Stage1.** Phase1 calibration is performed using block1 information.
- **Stage2.** Intermediate estimated totals are computed using phase1 output file and some of block2 information.
- **Stage3.** Phase2 calibration takes place where calibration weights for a two phase are computed.

Diagram1



Example

Suppose phase1 calibration is performed and calibration weights $w_{1k}, k \in s_1$ are obtained and for phase2 calibration, we have the following information:

Benchmark Constraints

We want to calibrate to three different types of totals:

- **Known** totals for variable x_1 by calibration group g_1 .
- **Known** totals for variable x_2 by calibration group g_2 (could be g_1 as well).

- **Estimated** totals using phase1 calculated calibration weights w_{1k} for variable x_3 by g_3 .
- **Estimated** totals using phase1 design/entry weights a_{1k} for variable x_4 by g_4 .

Range Restriction Constraints

We would like the calibration factors to satisfy, example: $0.5 \leq g_k \leq 1.2$, for all $k \in s_2$.

Minimization program

Minimize	$\sum_{k \in s_2} G_k(w_k, a_k^*) = \sum_{k \in s_2} \frac{c_k(w_k - a_k^*)^2}{a_k^*}, \text{ with respect to } w_k$
Subject to	- “Known totals” $\sum_{k \in s} w_k x_{1k} g_{1k} = T_{g_1}, \quad g_1 = 1, 2, \dots, G_1.$ - “Known totals” $\sum_{k \in s} w_k x_{2k} g_{2k} = T_{g_2}, \quad g_2 = 1, 2, \dots, G_2.$ - Estimated totals using w_{1k} $\sum_{k \in s} w_k x_{3k} g_{3k} = \hat{T}_{g_3} = \sum_{k \in s_1} w_{1k} x_{3k} g_{3k}, \quad g_3 = 1, 2, \dots, G_3.$ - Estimated totals using a_{1k} $\sum_{k \in s} w_k x_{4k} g_{4k} = \hat{T}_{g_4} = \sum_{k \in s_1} a_{1k} x_{4k} g_{4k}, \quad g_4 = 1, 2, \dots, G_4.$ - Range restrictions on factors $0.5 \leq w_k / a_k^* \leq 1.2$

Note

- The entry weight at phase2 is given by $a_k^* = a_{1k} a_{2k} = \frac{1}{\pi_k^*} = (\pi_{1k} \pi_{k|s_1})^{-1}$.
- g_4 , for instance, could be $g_1 \times g_3$ (province by Naics, e.g.)
- One might relax some of the benchmark constraints using the tolerance parameter.
- It is possible as well to specify a subset of sample units for which we do not want to modify entry weights.

2.3 Calibration for Cluster Sampling and Weight Integration

In (single-stage) cluster sampling, the population U is partitioned into sub-populations called clusters. The population of clusters is denoted here by U_I (sometimes denoted by U_c).

Examples

U : population of students;	U_I : clusters: schools.
U : population of persons;	U_I : clusters: households.
U : population of establishments;	U_I : clusters: enterprises/companies.

Cluster sampling is then defined as follows:

- A sample s_I of clusters (say, enterprises) is drawn from U_I using a given design. Sometimes denoted by s_c .
- Every element (say, establishment) in the selected clusters is observed.

We will use the letter i to denote a cluster and k for elements of a cluster. So the pair (i, k) will denote the element k (person/establishment) of cluster i (household/enterprise), for instance. Your typical scenario would be:

- We have a sample s_I of enterprises; say $E_1, \dots, E_n$ with corresponding design weights $a_1, \dots, a_n$.
- Goal: modify $a_1, \dots, a_n$ ("slightly") to obtain new weights $w_1, \dots, w_n$ subject to some constraints.
- Linear constraints on new weights of the form: $\sum_{k \in s_I} w_k \mathbf{x}_k = \sum_{U_I} \mathbf{x}_k = \mathbf{T}$.
- Constraints could be within (calibration) groups/domains.
- Here we minimize the following objective function, the so-called Chi-square (pseudo) distance

$$\sum_{i \in s_I} \frac{c_i (w_i - a_i)^2}{a_i}$$

where c_i is a positive adjustment factor.

Note

There are three possible scenarios when it comes to auxiliary information. They are detailed in the subsections below:

2.3.1 Auxiliary information at the element level only

We have clusters (enterprises/companies) and units/elements (establishments) within clusters. One setup would be to select a sample of clusters s_I , and then take all the units (establishments) within each selected cluster. Units within a cluster will be assigned the design/entry weight of the corresponding cluster. Here the sample file would look like the following:

Enterprise (cluster_Id)	Initial weight	Establishment (unit_Id)	Auxiliary variable (revenue)	Group variable (prov)	Group variable (Naics)	Other
E_1	a_1	e_{11}	x_{11}	ON	N_1	...
E_1	a_1	e_{12}	x_{12}	ON	N_3	...
E_1	a_1	e_{13}	x_{13}	AL	N_2	...
E_1	a_1	e_{14}	x_{14}	QC	N_1	...
E_1	a_1	e_{15}	x_{15}	QC	N_1	...
E_2	a_2	e_{21}	x_{21}	QC	N_2	...
E_2	a_2	e_{22}	x_{22}	QC	N_4	...
E_2	a_2	e_{23}	x_{23}	ON	N_5	...
E_2	a_2	e_{24}	x_{24}	ON	N_1	...
E_2	a_2	e_{25}	x_{25}	ON	N_2	...
E_2	a_2	e_{26}	x_{26}	ON	N_2	...
...	...	...	...	...	...	...

The problem

The same, modify entry weights $a_1, \dots, a_n$ to obtain calibration weights $w_1, \dots, w_n$. Calibration is done at the sampling level (enterprise in our example).

Issue

In majority of cases, calibration groups and auxiliary information are at the unit level only.

Example

We have the variable (total) revenue, x , for each establishment and we would like to calibrate by province and Naics (we know the total revenue by prov and Naics for the population of enterprises.)

Set-Up

- Let's use the letters i for clusters (enterprises) and k for elements (establishments) respectively.
- Let s_I denote the sample of clusters (enterprises).
- Let x be one available auxiliary variable at the element level, say revenue. So x_{ik} would be the revenue of establishment k belonging to enterprise i (of person k in household i .)
- Let the letter g denote a calibration group (PROV*NAICS, e.g.). So g_{ik} would be the indicator for the group, i.e., $g_{ik} = 1$ if establishment k of enterprise i belongs to the group g (e.g.: $ONT \times N_2$).

- Suppose we know the total T_g , in the population, for the variable x (revenue) for the calibration group g ; $\sum_{(i,k) \in U} x_{ik} g_{ik} = T_g$, the sum being taken over all the establishments of all the enterprises in the target population.

Two possible solutions here when it comes to retrieving the necessary information to build the calibration constraints:

Solution 1: create new auxiliary variables (lots of them) at the sampling level and calibrate as usual

Bring everything at the sampling level and then calibrate. This is done by creating new auxiliary variables which for each enterprise are the aggregate of the variable revenue (example) over all establishments of the corresponding enterprise for a given calibration group g ,

$$x_{g,i} = \sum_{k \in i} x_{ik} g_{ik}.$$

In that case, we would solve the following minimization problem

Minimize with respect to w_i Subject to	$\sum_{i \in S_I} \frac{c_i (w_i - a_i)^2}{a_i}$ - $\sum_{i \in S_I} w_i x_{g,i} = T_g, \quad g = 1, 2, \dots, G.$ - $l_i \leq w_i \leq u_i \quad \text{or} \quad L \leq \frac{w_i}{a_i} \leq M$
---	---

Issue

The number of auxiliary variables $x_{g,i}$ could be huge. Consequently, we will have to create very large data files. To avoid doing so, solution 2 below is available in **Gest_Calibration**.

Solution 2: use the file at the element level to build the constraints in [Optmodel](#) in SAS

We solve the same minimization problem with a difference; the constraints make use of variables at the establishment level (we read the file at the establishment level)

Minimize with respect to w_i Subject to	$\sum_{i \in S_I} \frac{c_i (w_i - a_i)^2}{a_i}$ - $\sum_{i \in S_I} \sum_{k \in i} w_i x_{ik} g_{ik} = \sum_{i \in S_I} w_i \sum_{k \in i} x_{ik} g_{ik} = T_g$ - $l_i \leq w_i \leq u_i \quad \text{or} \quad L \leq \frac{w_i}{a_i} \leq M$
---	---

Remarks

- So by doing so, one does not have to create a large number of the auxiliary variables $x_{g,i} = \sum_{k \in i} x_{ik} g_{ik}$ at the cluster level.
- **Gest_Calibration** reads the information at the file (lowest) level.

2.3.2 Auxiliary information at the cluster level only

This is the basic calibration problem with one level. The input sample file will look like the following (one line by cluster)

Enterprise (cluster_id)	Initial weight	Auxiliary variable (revenue)	Group variable (prov)	Group variable (Naics)	Other
E_1	a_1	x_1	ON	N_7	...
E_2	a_2	x_2	ON	N_3	...
E_3	a_3	x_3	AL	N_2	...
E_4	a_4	x_4	QC	N_1	...
E_5	a_5	x_5	ON	N_6	...
E_6	a_6	x_6	QC	N_2	...
E_7	a_7	x_7	BC	N_4	...
...	...	...	...	...	...

We solve the following mathematical programming problem:

Minimize with respect to w_i Subject to	$\sum_{i \in S_c} \frac{c_i (w_i - a_i)^2}{a_i}$ - $\sum_{i \in S_c} w_i z_i j_i = T_j, \quad j = 1, 2, \dots, J.$ - $l_i \leq w_i \leq u_i \quad \text{or} \quad L \leq \frac{w_i}{a_i} \leq M$
---	--

where auxiliary variables (z) and the calibration groups (j) are at the cluster level.

2.3.3 Auxiliary information at both the cluster and element levels

Here we suppose we have two sets of auxiliary variables; one at each level. We solve the following mathematical programming problem:

Minimize with respect to w_i	$\sum_{i \in S_c} \frac{c_i (w_i - a_i)^2}{a_i}$
Subject to	$\begin{aligned} & - \sum_{i \in S_c} w_i z_i j_i = T_j, \quad j = 1, 2, \dots, J. \\ & - \sum_{i \in S_c} \sum_{k \in i} w_i x_{ik} g_{ik} = \sum_{i \in S_c} w_i \sum_{k \in i} x_{ik} g_{ik} = T_g, \quad g = 1, 2, \dots, G. \\ & - l_i \leq w_i \leq u_i \quad \text{or} \quad L \leq \frac{w_i}{a_i} \leq M \end{aligned}$

where

- auxiliary variables (z) and the calibration groups (j) are at the cluster level;
- auxiliary variables (x) and the calibration groups (g) are at the element level.

2.4 Calibrating Bootstrap Replicate Weights

Gest_Calibration handles calibration of bootstrap replicate weights, that is calibrating more than one column of weights at a time. Say we have $B \geq 1$ bootstrap replicate weights (B columns of weights). For each bootstrap replicate $b = 1, \dots, B$, calibration is performed and an attempt is made to solve the following minimization problem:

The Problem

Minimize $G^{(b)}$ with respect to $w_i^{(b)}$ $b = 1, \dots, B$	$G^{(b)} = \sum_{i \in S} \frac{c_i (w_i^{(b)} - a_i^{(b)})^2}{a_i^{(b)}} \quad (1)$
Subject to	Benchmark constraints (calibration equations): $\sum_{i \in S} w_i^{(b)} \mathbf{x}_k = \sum_U \mathbf{x}_k \quad (2)$ Range restriction constraints: $l_i \leq w_i^{(b)} \leq u_i \quad \text{or} \quad L \leq \frac{w_i^{(b)}}{a_i^{(b)}} \leq M. \quad (3)$

- $a_i^{(b)}$: initial/entry Bootstrap replicate weight b (value for cluster i).
- $w_i^{(b)}$: calibrated Bootstrap replicate weight b (value for cluster i).

For each of the b replicates, the same calibration information is used; same sample file, distance, calibration equations and bounds.

Remark

The minimisation program above is written for $a_i^{(b)} > 0$. A modification of the distance and range restriction constraints (the factors) is necessary here when $a_i^{(b)} \leq 0$. For details, see [section 2.4.1](#) below.

2.4.1 Handling non-positive entry bootstrap replicates values $a_i^{(b)}$.

For some units, bootstrap initial weights could be zero or even negative. If $a_i^{(b)} \leq 0$, then in the distance function, $a_i^{(b)}$ is replaced by $\max(a_i^{(b)}, \varepsilon)$ ($\varepsilon = 10^{-6}$, for instance) and calibration is performed. **So negative values are replaced by a (“very small”) positive number in the distance function but not in the calibration equations.** This is a methodological choice. Another choice would be to force the negative initial/entry weights to take the value 1 (or other), for instance, and then calibrate.

New Chi-Square distance

$$G^{(b)} = \sum_{i \in S} \frac{c_i [w_i^{(b)} - \max(a_i^{(b)}, \varepsilon)]^2}{\max(a_i^{(b)}, \varepsilon)}$$

New Raking Ratio distance

$$G^{(b)} = \sum_{i \in S} c_i \left[w_i^{(b)} \log \left(\frac{w_i^{(b)}}{\max(a_i^{(b)}, \varepsilon)} \right) - w_i^{(b)} + \max(a_i^{(b)}, \varepsilon) \right].$$

Remark

- When $a_i^{(b)} > 0$, we obtain the usual distances programmed in Gest_Calibration version 1.
- We do not need to modify the values of the $a_i^{(b)} \leq 0$ in the SampleFile. So initial/entry weights (DesignWeightVariable) could be negative or zero in theory. However they are handled by the objective function (and in the range restriction constraints for the factors).

New Range Restriction Constraints for the factors

The factors constraints: $L \leq \frac{w_i^{(b)}}{a_i^{(b)}} \leq M$ will be replaced by the following $L \leq \frac{w_i^{(b)}}{\max(a_i^{(b)}, \varepsilon)} \leq M$,

$$L \times \max(a_i^{(b)}, \varepsilon) \leq w_i^{(b)} \leq M \times \max(a_i^{(b)}, \varepsilon).$$

Note

The user has to be cautious here in interpreting the factors when initial/entry weights are zero or negative.

2.4.2 Handling Special Cases

Calibrating $B = 1000$ replicates for instance could prove relatively time consuming if one relies on the OPTMODEL solver implemented in Gest_Calibration. Though the latter is relatively fast for most “reasonable” cases. Calibration in the case of a *single* partition (and one level) is done with a different algorithm; without the use of OPTMODEL. In the case of very large sample sizes, this would take few seconds compared to minutes in some cases!

2.5 Special Features and Exceptions

The module **Gest_Calibration** offers many interesting features when it comes to calibration. A summary of the different features is given in [section 2.1.6](#) above. For instance, one can

- Relax the benchmark constraints. This is done using a parameter called [tolerance](#) see section below;
- Specify a subset of initial weights not to be modified/calibrated. Take-alls, for instance;
- Avoid calibration in the case of small sample sizes in a given calibration group. User specifies the value for small;
- Obtain a report of empty calibration groups, missing values, auxiliary variable equals zero for all units in a calibration group and more.

Please refer to the user guide for more details about the available features in **Gest_Calibration**.

2.5.1 Tolerance Parameter

Instead of imposing equality constraints, we could loosen them by introducing a tolerance parameter.

Let x_1 be a given auxiliary variable, say revenue, and suppose that the corresponding total for calibration group g (say province=ON) is $T_{1g} \neq 0$ and let $S = \sum_{k \in S} w_k x_{1k} g_k$.

The constraints will then take the form:

$$\left| \frac{\sum_{k \in S} w_k x_{1k} g_k - T_{1g}}{T_{1g}} \right| \leq \varepsilon \quad \Leftrightarrow \quad T_{1g} - \varepsilon |T_{1g}| \leq S \leq T_{1g} + \varepsilon |T_{1g}|, \quad T_{1g} \neq 0.$$

$$\Leftrightarrow \quad (1 - \varepsilon)T_{1g} \leq S \leq (1 + \varepsilon)T_{1g}, \quad \text{if } T_{1g} > 0.$$

If $T_{1g} = 0$, one might set $S = 0$ (equality) or $-\varepsilon \leq S \leq \varepsilon$ or $-\varepsilon * A \leq S \leq \varepsilon * A$, $A > 0$. See J.N.K. Rao and A. C. Singh (1997), for example.

2.5.2 Fixed weights – Not to Calibrate Sets

Gest_Calibration allows the user to specify a subset of initial weights that he/she does not want to modify. A typical example would be the take-alls. This is achieved by using global parameter *CalibrationExclusionExpression*. The expression should correspond to a unique characteristic of the cluster (the level at which calibration is performed). This is important in the case of more than one level and calibration groups or information could be at a lower level (person/establishment in the case of households/enterprises.) It might happen that the user does not want to modify the weights of certain units. Here are some examples:

- Take-alls: we do not want to modify the weights of units having initial weight equals to 1.

- Do not change the weights of all units in a given domain; province=P.E.I., province=ON & SIZE=Medium, etc. (has to be unique within sampling units (clusters).)

The idea here is to leave those units defined in the exclude expression with their initial weight unchanged; design or other. In other words; we want to have $w_k = a_k$ for those units.

Calibration equation would look like (example):

$$\mathbf{a_1x_1 + a_2x_2} + w_3x_3 + w_4x_4 = T \quad (\text{bold part being constant; a sub total.})$$

The only variables here are w_3 and w_4 . It is like having

$$\mathbf{w_1x_1 + w_2x_2} + w_3x_3 + w_4x_4 = T$$

with the following additional constraints:

$$a_1 \leq w_1 \leq a_1, \quad a_2 \leq w_2 \leq a_2$$

Remarks

- If user provides a totals file with population totals excluding take-alls for instance, then a simple use of *CalibrationGroupExpression* parameter in *CalibrationGroupFile* suffices here. We do not need to use *CalibrationExclusionExpression* parameter.
- Calibration is done at the sampling level. In that case *CalibrationID* is the cluster ID which would be the Enterprise, for example. If the user is calibrating at the sampling level, he/she should put for *CalibrationExclusionExpression* something meaningful that relates to clusters/enterprises (same level) not some condition that relates to a lower level; unit/establishment level. The exclude expression must be on characteristics or properties of sampling units. Examples are:
 - The weights: since the design weight is constant within a cluster. So it makes sense to ask that companies with DesignWeight=1, have their Calibrated weights=1=DesignWeight
 - Number of employees in company, etc.
 - If group is PROV and we have only one level, then asking not to change the weights of PROV=PEI makes sense.
 - If group is PROV and we have more than one level, then we might end up with a problem if PROV is not constant within cluster.

2.5.3 Small Sample Size (Contributing Terms)

It is possible to remove a subset of benchmark equations from the set of all constraints if the number of sample units involved in an equation is less than a predetermined scalar. This can be achieved using parameter [SmallSampleSizeCalibration](#). See subsection [SmallSampleSizeCalibration](#) of the user guide.

Suppose, we had specified in the macro call: SmallSampleSizeCalibration1=5. Further suppose that all calibration groups had 6 units and more except for category 1 of group g_1 , i.e. $g_1 = 1$ (say a given stratum or province.) In that case, the benchmark constraints used in the minimization program would be the following (group starts at 2):

$$\sum_{k \in S_1} w_{1k} x_{1k} g_{1k} = T_{g_1}, \quad g_1 = 2, \dots, G_1.$$

Remarks

- Suppose we end up with few units in a given calibration group, say less than 5 (subjective), do we really want to calibrate in that case? If we have one unit for instance, do we really want to use the following equation $w_1 x_1 = T$? We might end up with a very large $w_1 = T/x_1$ ($x_1 \neq 0$).
- Suppose we have the following calibration equation:

$$w_1 x_1 + w_2 x_2 + w_3 x_3 + w_4 x_4 + w_5 x_5 + w_6 x_6 + w_7 x_7 = T,$$

and we decided that we do not calibrate if there are 3 sample units or less (subjective). Now the sample size in this case is $7 > 3$. We might be tempted to use this calibration equation. Now further assume that $x_4 = x_5 = x_6 = x_7 = 0$. In that case we will end up with the following equation:

$$w_1 x_1 + w_2 x_2 + w_3 x_3 = T,$$

and there are only three non-zero terms in the equation. Do we really want to calibrate in this case? So the idea is not calibrate if we end up with **contributing** (non-zero) terms less or equal to the specified value of the parameter SmallSampleSizeCalibration.

3 Using OPTMODEL in SAS

The module `Gest_Calibration` uses both the quadratic programming (QP) solver and the nonlinear programming (NLP) solver to solve the minimization program. These two solvers are part of the OPTMODEL procedure in SAS. Both use the so called *Interior Point Algorithm*. The default solver being QP. NLP is called if QP fails to establish convergence due, to numerical problems, example.

3.1 Quadratic Programming

A quadratic programming (QP) problem takes the following form (see [SAS/OR® 14.1 User's Guide \(2015\)](#), for example)

$$\begin{array}{ll} \min & \frac{1}{2} \mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{c}^T \mathbf{x} \\ \text{Subject to} & \mathbf{A} \mathbf{x} \{ \geq, =, \leq \} \mathbf{b} \\ & \mathbf{l} \leq \mathbf{x} \leq \mathbf{u} \end{array}$$

where

$\mathbf{Q} \in \mathbb{R}^{n \times m}$	is the quadratic (also known as Hessian) matrix
$\mathbf{A} \in \mathbb{R}^{m \times n}$	is the constraints matrix
$\mathbf{x} \in \mathbb{R}^n$	is the vector of decision variables
$\mathbf{c} \in \mathbb{R}^n$	is the vector of linear objective function coefficients
$\mathbf{b} \in \mathbb{R}^m$	is the vector of constraints right-hand sides (RHS)
$\mathbf{l} \in \mathbb{R}^n$	is the vector of lower bounds on the decision variables
$\mathbf{u} \in \mathbb{R}^n$	is the vector of upper bounds on the decision variables

This programming program is solved using the [Interior Point Algorithm](#) (IPA).

3.2 Interior Point Algorithm

For an overview of the IPA and the way it is implemented in SAS, see [SAS/OR® 14.1 User's Guide \(2015\)](#).

3.2.1 Excerpts from SAS User guide

The QP solver implements an infeasible primal-dual predictor-corrector interior point algorithm. To illustrate the algorithm and the concepts of duality and dual infeasibility, consider the following QP formulation (the primal):

$$\begin{array}{ll} \min & \frac{1}{2} \mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{c}^T \mathbf{x} \\ \text{Subject to} & \mathbf{A} \mathbf{x} \{ \geq, =, \leq \} \mathbf{b} \\ & \mathbf{l} \leq \mathbf{x} \leq \mathbf{u} \end{array}$$

The corresponding dual formulation is

$$\begin{array}{ll}
\max & -\frac{1}{2}\mathbf{x}^T\mathbf{Q}\mathbf{x} + \mathbf{b}^T\mathbf{y} \\
\text{Subject to} & -\mathbf{Q}\mathbf{x} + \mathbf{A}^T\mathbf{y} + \mathbf{w} = \mathbf{c} \\
& \mathbf{y} \geq \mathbf{0} \\
& \mathbf{w} \geq \mathbf{0}
\end{array}$$

Where $\mathbf{y} \in \mathbb{R}^m$ refers to the vector of dual variables and $\mathbf{w} \in \mathbb{R}^n$ refers to the vector of dual slack variables. The dual makes an important contribution to the certificate of optimality for the primal. The primal and dual constraints combined with complementarity conditions define the first-order optimality conditions, also known as KKT (Karush-Kuhn-Tucker) conditions, which can be stated as follows where $\mathbf{e} \equiv (1, \dots, 1)^T$ of appropriate dimension and $\mathbf{s} \in \mathbb{R}^m$ is the vector of primal *slack* variables:

$$\begin{array}{ll}
\mathbf{A}\mathbf{x} - \mathbf{s} & = \mathbf{b} \quad (\text{primal feasibility}) \\
-\mathbf{Q}\mathbf{x} + \mathbf{A}^T\mathbf{y} + \mathbf{w} & = \mathbf{c} \quad (\text{dual feasibility}) \\
\mathbf{W}\mathbf{X}\mathbf{e} & = \mathbf{0} \quad (\text{complementarity}) \\
\mathbf{S}\mathbf{Y}\mathbf{e} & = \mathbf{0} \quad (\text{complementarity}) \\
\mathbf{x}, \mathbf{y}, \mathbf{w}, \mathbf{s} & \geq \mathbf{0}
\end{array}$$

The letters $\mathbf{X}$, $\mathbf{Y}$, $\mathbf{W}$, and $\mathbf{S}$ denote matrices with corresponding x , y , w , and s on the main diagonal and zero elsewhere, as in the following example:

$$\mathbf{X} \equiv \begin{bmatrix} x_1 & 0 & \cdots & 0 \\ 0 & x_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & x_n \end{bmatrix}$$

If $(\mathbf{x}^*, \mathbf{y}^*, \mathbf{w}^*, \mathbf{s}^*)$ is a solution of the previously defined system of equations that represent the KKT conditions, then $\mathbf{x}^*$ is also an optimal solution to the original QP model.

At each iteration the interior point algorithm solves a large sparse system of linear equations,

$$\begin{bmatrix} \mathbf{Y}^{-1}\mathbf{S} & \mathbf{A} \\ \mathbf{A}^T & -\mathbf{Q} - \mathbf{X}^{-1}\mathbf{W} \end{bmatrix} \begin{bmatrix} \Delta\mathbf{y} \\ \Delta\mathbf{x} \end{bmatrix} = \begin{bmatrix} \Xi \\ \Theta \end{bmatrix}$$

where $\Delta\mathbf{x}$ and $\Delta\mathbf{y}$ denote the vector of *search directions* in the primal and dual spaces, respectively, and Θ and Ξ constitute the vector of the right-hand sides.

The preceding system is known as the reduced KKT system. The QP solver uses a preconditioned quasi-minimum residual algorithm to solve this system of equations efficiently.

Log File

When calibration is not successful, the log file from Gest_Calibration will contain the convergence status from the OPTMODEL call giving the reason for that. The convergence status is given by the variable *SOLUTION_STATUS*. The latter takes the following values:

SOLUTION_STATUS

indicates the solution status at termination. It can take one of the following values:

OPTIMAL	The solution is optimal.
CONDITIONAL_OPTIMAL	The optimality of the solution cannot be proven.
INFEASIBLE	The problem is infeasible.
UNBOUNDED	The problem is unbounded.
INFEASIBLE_OR_UNBOUNDED	The problem is infeasible or unbounded.
BAD_PROBLEM_TYPE	The problem type is unsupported by the solver.
ITERATION_LIMIT_REACHED	The maximum allowable number of iterations was reached.
TIME_LIMIT_REACHED	The solver reached its execution time limit.
FUNCTION_CALL_LIMIT_REACHED	The solver reached its limit on function evaluations.
INTERRUPTED	The solver was interrupted externally.
FAILED	The solver failed to converge, possibly due to numerical issues.

See the [SAS/OR® 14.1 User's Guide \(2015\)](#) for more details on the OPTMODEL procedure.

REFERENCES

Deville, J.-C., and Särndal, C.-E. (1992). *Calibration estimators in survey sampling*. Journal of the American Statistical Association, 87, 376-382.

Estevao, V. and Särndal, C.-E. (2009). *A new face on two-phase sampling with calibration estimators*. Survey Methodology, **35**(1), 3-14.

SAS Institute Inc. 2015. *SAS/OR® 14.1 User's Guide: Mathematical Programming*. Cary, NC: SAS Institute Inc.

J.N.K. Rao and A. C. Singh (1997) *A ridge-shrinkage method for range-restricted weight calibration in survey sampling*

APPENDIX A (macro call)

```
%Gest_Calibration(
    CalibrationPhases=,
    CalibrationID=,
    Epsilon=,
    Report=,
    ExclusionFileBypass=,

/*** PHASE 1 CALIBRATION INFO BLOCK ***/
    SampleFile1=,
    WeightFile1=,
    DesignWeightVariable1=,
    AdjustmentFactorVariable1=,
    CalibrationGroupFile1=,
    ExclusionFileBypass1=,
    Distance1=,
    WeightBoundsFile1=,
    LowerWeightBound1=,
    UpperWeightBound1=,
    LowerFactorBound1=,
    UpperFactorBound1=,
    CalibrationExclusionExpression1=,
    SmallSampleSizeCalibration1=,
    ExclusionFile1=,
    ResolvedGroupsFile1=,
    OutputFile1=,

/*** PHASE 2 CALIBRATION INFO BLOCK ***/
    SampleFile2=,
    WeightFile2=,
    DesignWeightVariable2=,
    AdjustmentFactorVariable2=,
    CalibrationGroupFile2=,
    ExclusionFileBypass2=,
    Distance2=,
    WeightBoundsFile2=,
    LowerWeightBound2=,
    UpperWeightBound2=,
    LowerFactorBound2=,
    UpperFactorBound2=,
    CalibrationExclusionExpression2=,
    SmallSampleSizeCalibration2=,
    ExclusionFile2=,
    ResolvedGroupsFile2=,
    OutputFile2=,

/*** BOOTSTRAP CALIBRATION INFO BLOCK ***/
    NumberOfReplicates=,
    InputBootstrapFile=,
```

```

InputWeightPrefix=,
InputBootstrapStatusFile=,
OutputBootstrapFile=,
OutputWeightPrefix=,
OutputBootstrapStatusFile=,

```

```

/**** PARALLEL MODE AND EXECUTION OPTIONS ****/
ParallelMode=,
NumberOfSessions=,
SASApplicationServer=,
MacroExecutionMode=,
GestOptions=);

```

List of parameters with short description

Parameter name	Short description
AdjustmentFactorVariable	This is the (regression) adjustment weight c_{1k} and c_{2k} resp.
CalibrationExclusionExpression	This parameter allows the user to provide a set of units for which he/she does not want to modify the initial weights.
CalibrationGroupFile	Input file. A SAS data file containing the necessary information to create the calibration equations/constraints.
CalibrationID	Unique identifier. The level at which calibration is performed.
CalibrationPhases	This parameter determines whether we are performing one or two-phase calibration. For this version only one-phase calibration is permitted when calibrating bootstrap replicates.
DesignWeightVariable	This is the design initial/entry weight of the corresponding phase; a_{1k} or a_{2k}
Distance	This is the distance used in the minimization program.
Epsilon	A (small) positive number. Epsilon would replace the negative or zero initial/entry weight <u>in the distance function</u> .
ExclusionFile	Output file. A SAS data file containing a list of excluded calibration groups.
ExclusionFileBypass	This parameter allows the user to skip/stop the calibration

	procedure or use some calibration equations in the case of missing values or other limitations to calibration
GestOptions	Option string for extra G-Est diagnostics, mainly for log content.
InputBootstrapFile	Input file. A SAS-data-set. Must contain the Bootstrap replicates and the variables (columns) defining CalibrationID. These replicates are the initial/entry weights for the calibration procedure.
InputBootstrapStatusFile	Input file containing the status for each replicate weight; that is to be used or not.
InputWeightPrefix	Prefix of the bootstrap replicates names.
LowerFactorBound	A uniform lower constraint on all sought calibration factors.
LowerWeightBound	A uniform lower constraint on all sought calibration weights
MacroExecutionMode	Used to validate parameters and/or run the macro.
NumberOfSessions	When the parallel mode is used, NumberOfSessions corresponds to the number of SAS sessions the user wants to dedicate for a given call to Gest_Calibration.
NumberOfReplicates	Number of replicates to be used in procedure. The file may contain $B = 1000$ replicates, but the user may decide to use only the first 500, for instance.
OutputBootstrapFile	Output file at the CalibrationID level containing the calibrated bootstrap replicates.
OutputBootstrapStatusFile	Output file containing the status for each replicate weight after the calibration procedure. Those replicates for which the calibration procedure was successful have their status set to 'OK', otherwise it is set to 'CAL'.
OutputWeightPrefix	Prefix for the output calibrated bootstrap replicates names.
OutputFile	A SAS data file containing the unique identifier and the calculated calibration weights and factors
ParallelMode	Option describing the type of parallel processing to use. Parallel processing allows the use of different cores of a computer chip or different computers at the same time to perform various computations.
ResolvedGroupsFile	Output file indicating which calibration groups were

	successfully calibrated (when simple calibration is done).
SampleFile	Input file. A SAS data file. Must contain at least the following quantities: CalibrationID.
SASApplicationServer	Name of the server to use (the SAS Grid in our case.)
SmallSampleSizeCalibration	When specified, the log file will include a list of all calibration groups with a sample size less or equal to the specified value and calibration won't take place for those groups.
UpperFactorBound	A uniform upper constraint on all sought calibration factors.
UpperWeightBound	A uniform upper constraint on all sought calibration weights.
WeightBoundsFile	Input file. A SAS data file containing individual lower and upper bound constraints for the sought calibration weights.
WeightFile	Input file. A SAS data file. Must contain at least the following quantities: CalibrationId, DesignWeightVariable.

APPENDIX B

B1 Using Single-Stage Calibration to Perform Two-Stage Calibration With Integrated Weighting

In two-stage sampling, one would subsample from each selected cluster at first-stage instead of taking all units from each selected cluster. Let

s_i	the selected sample of units in selected cluster i : $s_i \subseteq C_i$.
$\pi_{k i}$	Conditional inclusion probability of unit k given that cluster i is selected.
$a_{k i} = 1/\pi_{k i}$	Second-stage conditional design weight
$a_k = a_{Ii}a_{k i}$	overall design weight of unit k in selected cluster i .
$a_{Ii} = 1/\pi_{Ii}$	Design weight for cluster i .
w_{Ii}	Calibrated weights for cluster i .
$w_k = w_{Ii}a_{k i}$	overall calibrated weight of unit k in selected cluster i .
$\mathbf{z}_i$	Vector of auxiliary variables of cluster $i \in s_I$.
$\sum_{i \in U_I} \mathbf{z}_i$	Corresponding total over the population of clusters.
$\mathbf{x}_{ik}, k \in i$	Vector of auxiliary variables for unit k in cluster $i \in s_I$.
$\sum_{i \in U_I} \sum_{k \in i} \mathbf{x}_{ik}$	Corresponding vector of totals over the population of units.

Calibration

Here we suppose we have two sets of auxiliary variables, one at each level and we know the corresponding totals over the whole population or calibration groups. These totals could be estimated as well like in the case of a two-phase.

Problem

Find calibration weights w_{Ii} and w_k at the cluster and unit levels respectively satisfying certain benchmark (and possibly range restriction) constraints with the following condition linking the two

$$w_k = w_{Ii}a_{k|i}, \quad k \in i.$$

We need to solve the following mathematical programming problem:

Minimize	$\sum_{i \in S_c} \frac{c_i(w_{Ii} - a_i)^2}{a_i}$	with respect to w_i
Subject to	$- \sum_{i \in S_c} w_{Ii} \mathbf{z}_i j_i = \sum_{i \in U_I} \mathbf{z}_i j_i, \quad j = 1, 2, \dots, J.$ $- \sum_{i \in S_c} \sum_{k \in \mathbf{s}_i} w_k \mathbf{x}_{ik} g_{ik} = \sum_{i \in S_c} w_{Ii} \sum_{k \in \mathbf{s}_i} a_{k i} \mathbf{x}_{ik} g_{ik} = \sum_{i \in U_I} \sum_{k \in i} \mathbf{x}_{ik} g_{ik}, \quad g = 1, 2, \dots, G.$ $- l_i \leq w_{Ii} \leq u_i \quad \text{or} \quad L \leq \frac{w_{Ii}}{a_{Ii}} \leq M$	

- auxiliary vector ($\mathbf{z}$) and the calibration groups (j) are at the cluster level;
- auxiliary vector ($\mathbf{x}$) and the calibration groups (g) are at the unit level;
- $j_i = I(i \in j); \quad g_{ik} = I((i, k) \in g).$

Remarks

- We have the following equation

$$\begin{aligned} \sum_{k \in \mathbf{s}} w_k \mathbf{x}_k^* g_k &= \sum_{i \in S_c} \sum_{k \in \mathbf{s}_i} w_k \mathbf{x}_{ik} g_{ik} = \sum_{i \in S_c} w_{Ii} \sum_{k \in \mathbf{s}_i} a_{k|i} \mathbf{x}_{ik} g_{ik} = \sum_{i \in S_c} w_{Ii} \hat{\mathbf{x}}_{gi} = \sum_{i \in U_I} \sum_{k \in i} \mathbf{x}_{ik} g_{ik} \\ &= \sum_{k \in U} \mathbf{x}_k^* g_k \end{aligned}$$

So aggregate ("estimated") auxiliary variables $\hat{\mathbf{x}}_{gi}$ at the cluster level could be created ...

- $\mathbf{x}_k^*$ is the vector of auxiliary variables for unit k .

In the following, we show how to use Gest_Calibration to calibrate in the case of a two-stage design.

Solution using single-stage calibration

By defining a new set of auxiliary variables, say $\mathbf{x}_{ik}^* = a_{k|i} \mathbf{x}_{ik}$, the minimization program becomes:

Minimize	$\sum_{i \in S_c} \frac{c_i(w_{Ii} - a_i)^2}{a_i}$	with respect to w_i
Subject to	$- \sum_{i \in S_c} w_{Ii} \mathbf{z}_i j_i = \sum_{i \in U_I} \mathbf{z}_i j_i, \quad j = 1, 2, \dots, J.$ $- \sum_{i \in S_c} \sum_{k \in \mathbf{s}_i} w_k \mathbf{x}_{ik} g_{ik} = \sum_{i \in S_c} w_{Ii} \sum_{k \in \mathbf{s}_i} \mathbf{x}_{ik}^* g_{ik} = \sum_{i \in U_I} \sum_{k \in i} \mathbf{x}_{ik} g_{ik}, \quad g = 1, 2, \dots, G.$ $- l_i \leq w_{Ii} \leq u_i \quad \text{or} \quad L \leq \frac{w_{Ii}}{a_{Ii}} \leq M$	

Remarks

- The total we specify in CalibrationGroupFile is $\sum_{i \in U_I} \sum_{k \in i} \mathbf{x}_{ik} g_{ik}$.
- $\mathbf{s}_i$ is regarded as a take-all sample. This makes no difference for the module.
- w_{Ii} = cluster calibrated weight.
- $w_k = w_{Ii} a_{k|i}$ = the unit calibrated weight. These weights are obtained from the SampleFile containing $a_{k|i}$ and the CalibrationWeightFile containing w_{Ii} .

-

B2 Calibration (Household – Person – Kish individual)

Social surveys where households and persons are interviewed.

The Setting

- **A sample s_H of households**

- Household $h \in s_H$ has weight $a_h = \frac{1}{\pi_h}$

- Auxiliary variables/vector z_h with totals T_z .

Calibration Equation (by calibration group categories j):

$$\sum_{h \in s_H} w_h z_h j_h = T_z = \sum_{h \in U_H} z_h j_h, \quad j = 1, 2, \dots, J.$$

- **A sample s_I of individuals**

- All members i in household h (sample s_I), have weight $a_{h,i} = a_h$.

- Auxiliary variables/vector $x_{h,i}$, with totals T_x

Calibration Equation (by calibration group categories g):

$$\sum_{h \in s_H} \sum_{i \in h} w_h x_{hi} g_{hi} = T_x = \sum_{h \in U_H} \sum_{i \in h} x_{hi} g_{hi}, \quad g = 1, 2, \dots, G.$$

Note: $\sum_{h \in s_H} \sum_{i \in h} w_h x_{hi} g_{hi} = \sum_{h \in s_H} w_h x_{h,g}$ where $x_{h,g} = \sum_{i \in h} x_{hi} g_{hi}$.

- **A sample s_k of (Kish-) individuals.**

- One (Kish-) individual k_h in each household h , selected with a SRS among the e_h "eligible" members (sample s_k). Has weight $d_{k_h} = e_h a_h$

- Auxiliary variables/vector v_{k_h} , with totals T_v

Calibration Equation (by calibration group categories l):

$$\sum_{h \in s_H} \sum_{k_h \in s_k \cap h} w_h e_h v_{k_h} l_{hi} = \sum_{h \in s_H} w_h e_h v_{k_h} l_{hi} = T_v, \quad l = 1, 2, \dots, L.$$

Objectives

- to produce the same weights for all members of a household
- to ensure consistency in the statistics obtained for the various data files

Method: we calculate for each household h :

- the **totals** of the individuals-variables: $z_h = \sum_{i \in h} z_{h,i}$
 - the **estimated totals** of the Kish individuals variables: $\hat{v}_h = e_h v_{k_h}$
- Vector of calibration variables for household h : $(x_h, z_h, \hat{v}_h)$
 Vector of totals: (T_x, T_z, T_v)

Calibration equations for the sample of households (omitting the groups):

$$\sum_{h \in s_H} w_h \times (x_h, z_h, \hat{v}_h) = (T_x, T_z, T_v).$$

→ calibration weights:	$w_h,$	$w_{h,i} = w_h,$	and	$w_{k_h} = e_h w_h.$
------------------------	--------	------------------	-----	----------------------

Minimization Program

Minimize	$\sum_{h \in S_H} \frac{c_h(w_h - a_h)^2}{a_h}$	with respect to	w_h
Subject to	• $\sum_{h \in S_H} w_h z_h j_h = T_z, \quad j = 1, 2, \dots, J.$ • $\sum_{h \in S_H} \sum_{i \in h} w_h x_{hi} g_{hi} = T_x, \quad g = 1, 2, \dots, G.$ • $\sum_{h \in S_H} \sum_{k_h \in h} w_h \hat{v}_{k_h} I(k_h \in s_k) l_{hi} = \sum_{h \in S_H} w_h \hat{v}_h l_{hi} = T_v, \quad l = 1, 2, \dots, L.$ • $l_h \leq w_h \leq u_h \quad \text{or} \quad L \leq \frac{w_h}{a_h} \leq M$		

Remarks

- For the Kish-sample, the user might consider the two following options:
 - Read auxiliary information at the cluster/household level by creating the auxiliary variables $\hat{v}_h$ and calibrate (simple option.)
 - Read auxiliary information at the unit/individual level by creating the auxiliary variables $\hat{v}_{k_h} = e_h v_{k_h} I(k_h \in s_k)$ and calibrate.
- User is supposed to enter with constant within clusters/households weights, like doing a weight sharing first.